# Case Study 2: Technical Presentation

## STAT 946 – Case Studies in Data Science

Dario Greco  Lovera Lokeswara  Sanika Poojary  Sophia Feng

2026-03-09

Section 1

# Complex model: N-HiTS

# Why N-HiTS

## Definition

N-HiTS: Neural Hierarchical Interpolation for Time Series

Goal: capture nonlinearities and multi-scale temporal structure with a deep, compute-heavy model.

Why it fits air-quality data:

- multi-resolution learning (slow trends + fast spikes)
- non-linear mapping from weather and seasonality to $PM_{2.5}$
- residual learning to decompose signal into components

Input and output shapes:

$$x_t \in \mathbb{R}^{L \times d}, \quad d = 11$$

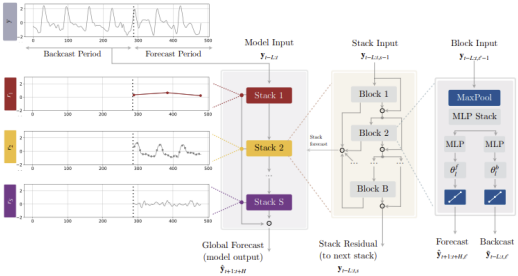
$$\hat{y}_{t+1:t+24} = f_\theta(x_t), \qquad \hat{y}_{t+4} = [\hat{y}_{t+1:t+24}]_4$$

- lookback: $L \in \{24, 48\}$
- horizon head: 24-step output, but we report the 4-hour-ahead value

# N-HiTS Architecture: Hierarchical multi-resolution

The N-HITS model as "3 frequency stacks":

- Stack 1 (macro) → Block 1: max-pool ×4, $n_\theta = 6$
- Stack 2 (mid) → Block 2: max-pool ×2, $n_\theta = 12$
- Stack 3 (fine) → Block 3: no pooling, $n_\theta = 24$



# Our N-HiTS structure

Three frequency blocks:

| Block | Pool size | Resolution captured | Forecast knots |
|---|---|---|---|
| Block 1 | 4 | long-term trends | 6 |
| Block 2 | 2 | medium variation | 12 |
| Block 3 | 1 | short-term spikes | 24 |

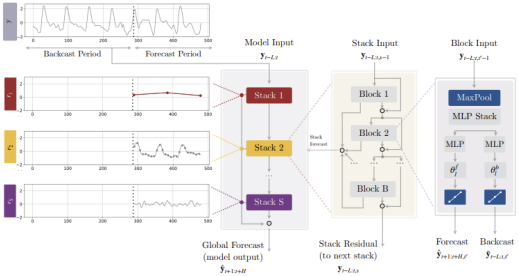
Forecast combination:

- each block produces a forecast
- forecasts are summed
- we evaluate the **4-hour-ahead prediction**

# How one N-HiTS block works

Pipeline inside each block:

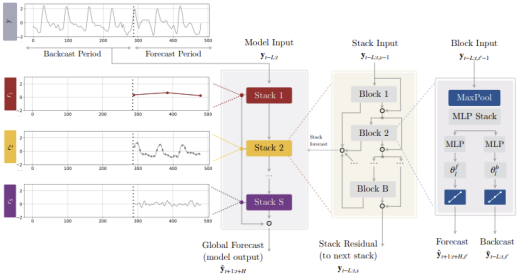
1. **Pooling (multi-scale view)**
   Compress the input window to capture patterns at a specific time scale.
   - Block 1: coarse trends (pool $\times 4$)
   - Block 2: medium variation (pool $\times 2$)
   - Block 3: fine detail (pool $\times 1$)
2. **MLP encoder**
   A small neural network learns a hidden representation of the time window.
3. **Backcast (signal explanation)**
   The block reconstructs the part of the input signal it can explain.



# How one N-HiTS block works

Pipeline inside each block:

4. **Residual update**
   The explained signal is removed so the next block focuses on what remains.
5. **Forecast generation**
   The block predicts a small set of forecast knots.
6. **Interpolation to horizon**
   These knots are linearly interpolated to produce a 24-step forecast.




Final prediction:

$$\hat{y} = \hat{y}^{(1)} + \hat{y}^{(2)} + \hat{y}^{(3)}$$

Each block contributes a forecast at a different temporal resolution.

# Target-specific residual learning

Split features:

- target: $PM_{2.5}$
- exogenous: weather + time features

Residuals are applied only to the target channel:

$$r^{(1)} = y_{\text{target}} - \widehat{x}^{(1)}_{\text{target}}$$

For block 2 and 3, the input is:

$$x^{(k)} = \left[r^{(k-1)},\ \text{exog}\right]$$

Final forecast is the sum of all block forecasts:

$$\hat{y}_{t+1:t+24} = \widehat{y}^{(1)} + \widehat{y}^{(2)} + \widehat{y}^{(3)}, \qquad \hat{y}_{t+4} = \left[\hat{y}_{t+1:t+24}\right]_4$$

# Training and evaluation

Data:

- standardized 11 features ($PM_{2.5}$ + weather + seasonal sin/cos)
- split: 70% train, 15% validation, 15% test (time-ordered)

Optimization:

- loss: MSE
- optimizer: Adam
- epochs: up to 50
- early stopping: patience = 8 (best val RMSE checkpoint)

Metrics:

- RMSE on test set for 4-hour-ahead prediction

Compute:

- GPU (Waterloo Remote Server), batch size 128
- runtime per config $\sim$ 30 mins

# Hyperparameter tuning

We run a small hyperparameter sweep centered on best-performing configs:

- lookback $L \in \{24, 48\}$
- hidden width $\in \{128, 256, 512\}$
- learning rate $\in \{0.001, 0.0005, 0.0001, 0.00005\}$
- dropout $\in \{0.1\}$

Selection rule:

- choose lowest validation RMSE
- trainable parameters: 1,453,890 parameters

**Chosen config**

$L = 24$, hidden width $= 256$, learning rate $= 0.0001$, dropout $= 0.1$